

CyberVault — Guía Integral de Instalación del Plugin Chrome

Código: CV-DOC-INST-001
Versión: 1.0.0
Fecha: 2026-09-17
Estado: Verificado con servicios levantados

Índice

- 1. [Prerrequisitos](#)
- 2. [Arquitectura del Sistema](#)
- 3. [Instalación de Servicios Backend](#)
- 4. [Verificación de Servicios](#)
- 5. [Construcción del Plugin Chrome](#)
- 6. [Instalación del Plugin en Chrome](#)
- 7. [Configuración del Plugin](#)
- 8. [Uso Básico](#)
- 9. [Troubleshooting](#)
- 10. [Variables de Entorno](#)

1. Prerrequisitos

1.1 Software Requerido

Componente	Versión Mínima	Versión Verificada	Propósito
Node.js	18.x	20.x	Runtime para build de la extensión
npm	9.x	10.x	Gestor de paquetes
Docker	24.x	26.x	Contenedores de servicios
Docker Compose	v2.x	v2.29	Orquestación de servicios
Google Chrome	120+	—	Navegador para la extensión

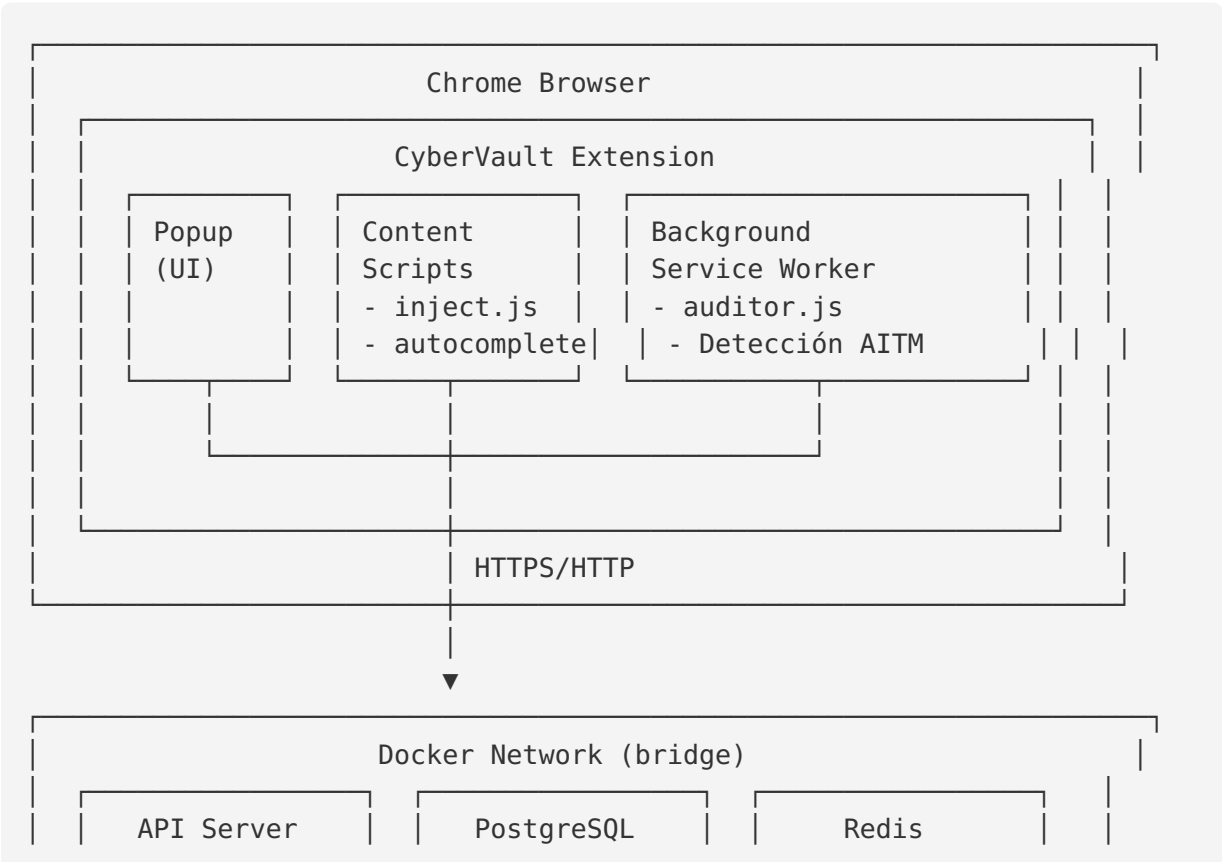
1.2 Hardware Mínimo

Recurso	Mínimo	Recomendado
RAM	4 GB	8 GB
Disco	2 GB libre	5 GB libre
CPU	2 cores	4 cores

1.3 Puertos Requeridos

Puerto	Servicio	Protocolo
3010	CyberVault API	HTTP
5432	PostgreSQL	TCP (interno Docker)
6379	Redis	TCP (interno Docker)

2. Arquitectura del Sistema



	(Node.js) Puerto: 3000 Host: 3010	(16-alpine) Puerto: 5432 Interno	(7-alpine) Puerto: 6379 Interno	
--	---	--	---------------------------------------	--

3. Instalación de Servicios Backend

3.1 Clonar el Repositorio

```
git clone <URL_DEL_REPOSITORIO>
cd cybervault
```

3.2 Configurar Variables de Entorno

```
# Copiar plantilla de configuración
cp .env.example .env

# Editar con valores específicos del entorno
nano .env
```

Variables mínimas requeridas en `.env` :

```
# PostgreSQL
POSTGRES_PASSWORD=<contraseña_segura>

# Redis (opcional, tiene default)
REDIS_PASSWORD=cybervault_redis_secret

# JWT (opcional para desarrollo)
JWT_SECRET=<tu_jwt_secret>

# API Port (default: 3000, mapeado a 3010 en host)
API_PORT=3010
```

3.3 Levantar Servicios

```
# Opción 1: Solo servicios core (recomendado para desarrollo)
docker compose up -d postgres redis api
```

```
# Opción 2: Stack completo con documentación
docker compose --profile docs up -d

# Opción 3: Stack completo con IPFS
docker compose --profile ipfs --profile docs up -d
```

3.4 Verificar Estado de Contenedores

```
docker compose ps
```

Salida esperada:

NAME	IMAGE	STATUS	PORTS
cybervault-api	cybervault-api	Up (healthy)	0.0.0.0:3010->3000/tcp
cybervault-db	postgres:16-alpine	Up (healthy)	5432/tcp
cybervault-redis	redis:7-alpine	Up (healthy)	6379/tcp

4. Verificación de Servicios

4.1 Health Check

```
curl -s http://localhost:3010/health | python3 -m json.tool
```

Respuesta esperada:

```
{
  "status": "healthy",
  "timestamp": "2026-09-17T23:45:32.338Z",
  "service": "cyber-vault-api",
  "checks": {
    "database": "ok",
    "ipfs": "not_configured"
  },
  "metrics": {
    "uptimeSeconds": 160419,
    "totalRequests": 5,
    "errorRate": 0
  }
}
```

4.2 Ready Check

```
curl -s http://localhost:3010/ready | python3 -m json.tool
```

Respuesta esperada:

```
{
  "status": "ready",
  "timestamp": "2026-09-17T23:45:32.779Z",
  "checks": {
    "database": "ok",
    "ipfs": "not_configured"
  }
}
```

4.3 Verificación de Base de Datos

```
# Entrar al contenedor de PostgreSQL
docker exec -it cybervault-db psql -U cybervault -d cybervault

# Verificar tablas
\dt

# Verificar conexión
SELECT 1 AS test;

# Salir
\q
```

4.4 Verificación de Redis

```
# Entrar al contenedor de Redis
docker exec -it cybervault-redis redis-cli -a cybervault_redis_secret

# Verificar conexión
PING

# Salir
EXIT
```

5. Construcción del Plugin Chrome

5.1 Instalar Dependencias

```
npm install
```

5.2 Construir la Extensión

```
npm run build:ext
```

Salida esperada:

```
✓ Background script bundled
✓ Popup built
✓ Content script built
✓ Autocomplete content script built
✓ Options page built
✓ Icons copied
✓ Static files copied
✓ Build complete!
```

5.3 Estructura del Build

```
dist/
├── manifest.json          # Manifest V3 de Chrome
├── background/
│   └── auditor.js         # Service Worker (detección AITM)
├── ui/
│   ├── popup/
│   │   ├── popup.html    # Interfaz principal
│   │   ├── popup.js      # Lógica del popup
│   │   └── popup.css      # Estilos
│   ├── options/
│   │   ├── options.html  # Página de configuración
│   │   ├── options.js    # Lógica de opciones
│   │   └── options.css   # Estilos
│   └── content-scripts/
│       ├── inject.js     # Inyección en páginas
│       └── autocomplete.js # Autocompletado seguro
├── icons/
│   ├── icon-16.png       # Icono 16x16
│   └── icon-48.png       # Icono 48x48
```

```
| icon-128.png      # Icono 128x128
| [archivos .js.map] # Source maps (desarrollo)
```

6. Instalación del Plugin en Chrome

6.1 Modo Desarrollador

1. Abrir Chrome y navegar a `chrome://extensions/`
2. Habilitar **"Modo desarrollador"** (toggle en la esquina superior derecha)
3. Hacer clic en **"Cargar extensión sin empaquetar"** (Load unpacked)
4. Seleccionar la carpeta `dist/` del proyecto: `/home/search14/cybervault/dist/`
5. La extensión aparecerá en la barra de herramientas con el icono de CyberVault

6.2 Verificar Instalación

1. Hacer clic en el icono de CyberVault en la barra de herramientas
2. Debería mostrarse el popup con el mensaje "Vault is locked"
3. Verificar en `chrome://extensions/` que no hay errores

6.3 Permisos de la Extensión

Permiso	Propósito
<code>storage</code>	Almacenamiento local de configuración y vault
<code>alarms</code>	Verificación periódica de dominios
<code>tabs</code>	Acceso a información de pestañas para detección AITM
Host: <code>api.pwnedpasswords.com</code>	Verificación de contraseñas comprometidas
Host: <code>otx.alienvault.com</code>	Threat intelligence
Host: <code>api.github.com</code>	Actualizaciones y verificación

7. Configuración del Plugin

7.1 Página de Configuración

1. Hacer clic derecho en el icono de CyberVault 2. Seleccionar **"Opciones"** (Options)
2. Configurar dominios confiables

7.2 Dominios Confiables

Los dominios confiables se saltan la validación AITM:

```
ejemplo.com  
gmail.com  
github.com
```

7.3 Funcionalidades Disponibles

Función	Descripción
Vault Lock/Unlock	Bloquear/desbloquear vault con passphrase
Generación de Credenciales	Generar usuario/contraseña seguros por dominio
Detección AITM	Alerta en tiempo real de ataques AITM
Autocompletado	Relleno seguro de formularios
Verificación HIBP	Chequeo contra Have I Been Pwned

8. Uso Básico

8.1 Flujo de Primer Uso

1. Instalar extensión (ver Sección 6)
2. Abrir cualquier sitio web
3. Hacer clic en icono CyberVault
4. Establecer passphrase maestra
5. El vault se crea localmente (zero-knowledge)
6. Generar credenciales para el dominio actual
7. Las credenciales se almacenan cifradas

8.2 Generación de Credenciales

```
// La extensión genera credenciales con:  
// - Salt único por dominio  
// - Pepper de 128 bits  
// - Validación de entropía  
// - Almacenamiento AES-256-GCM
```

8.3 Detección AITM

El background service worker (`auditor.js`) ejecuta:

1. **Validación de dominio** — exact match, Unicode confusables
2. **Typosquatting** — distancia de Levenshtein
3. **Integridad DOM** — fingerprinting de contenido
4. **Análisis de timing** — detección de proxies
5. **Seguridad de cookies** — flags de seguridad

9. Troubleshooting

9.1 Errores Comunes

Error	Causa	Solución
<code>password authentication failed</code>	PostgreSQL no acepta la contraseña	Verificar <code>POSTGRES_PASSWORD</code> en <code>.env</code> y reiniciar DB
<code>Extension failed to load</code>	Manifest inválido o archivos faltantes	Re-ejecutar <code>npm run build:ext</code>
<code>Cannot connect to API</code>	Servidor API no está corriendo	<code>docker compose up -d api</code>
<code>Vault is locked persiste</code>	Passphrase incorrecta	Reiniciar vault (borrar datos de storage)

9.2 Logs de Servicios

```
# Logs de la API  
docker compose logs -f api  
  
# Logs de PostgreSQL  
docker compose logs -f postgres
```

```
# Logs de Redis
docker compose logs -f redis

# Logs recientes (últimas 100 líneas)
docker compose logs --tail 100 api
```

9.3 Reiniciar Servicios

```
# Reiniciar solo la API
docker compose restart api

# Reiniciar todo
docker compose down && docker compose up -d

# Reiniciar con reconstrucción
docker compose up -d --build api
```

9.4 Limpiar Datos

```
# △ ELIMINA TODOS LOS DATOS
docker compose down -v

# Limpiar solo Redis
docker compose exec redis redis-cli FLUSHALL
```

10. Variables de Entorno

10.1 Referencia Completa

Variable	Default	Descripción
PORT	3000	Puerto del servidor API
API_PORT	3000	Puerto mapeado en host
NODE_ENV	development	Entorno de ejecución
JWT_SECRET	(ninguno)	Secreto para JWT (sin él, auth deshabilitada)
USE_POSTGRES	false	Usar PostgreSQL en vez de memoria

Variable	Default	Descripción
DATABASE_URL	—	URL de conexión PostgreSQL
REDIS_URL	redis://localhost:6379	URL de conexión Redis
REDIS_PASSWORD	cybervault_redis_secret	Contraseña Redis
POSTGRES_PASSWORD	—	Contraseña PostgreSQL
IPFS_API_URL	http://localhost:5001	Endpoint API de IPFS
IPFS_ENABLED	false	Habilitar IPFS
HIBP_ENABLED	true	Verificar contra Have I Been Pwned
RATE_LIMIT_MAX	100	Máximo de requests por ventana
RATE_LIMIT_WINDOW_MS	900000	Ventana de rate limit (15 min)
HTTPS_ENABLED	false	Habilitar HTTPS con TLS
TLS_CERT_PATH	./certs/server.crt	Ruta del certificado TLS
TLS_KEY_PATH	./certs/server.key	Ruta de la clave TLS

11. Endpoints de la API

Método	Endpoint	Descripción
GET	/health	Health check
GET	/ready	Readiness check
POST	/api/v1/auth/register	Registrar usuario
POST	/api/v1/auth/login	Login
GET	/api/v1/auth/verify	Verificar token JWT
POST	/api/v1/vaults	Crear vault
GET	/api/v1/vaults	Listar vaults
GET	/api/v1/vaults/:id	Obtener vault

Método	Endpoint	Descripción
DELETE	/api/v1/vaults/:id	Eliminar vault
POST	/api/v1/credentials/generate	Generar credenciales
POST	/api/v1/credentials/extract	Extraer credenciales
GET	/api/v1/credentials/validate	Validar formato
GET	/api	Info de la API

12. Acceso por Red

12.1 Acceso Local (misma máquina)

API: <http://localhost:3010>
 Health: <http://localhost:3010/health>
 Ready: <http://localhost:3010/ready>
 Swagger: <http://localhost:3010/api/docs>

12.2 Acceso por LAN

API: <http://10.9.9.114:3010>
 Health: <http://10.9.9.114:3010/health>

12.3 Acceso por Tailscale (remoto)

API: <http://100.78.47.108:3010>
 Health: <http://100.78.47.108:3010/health>

13. Seguridad

13.1 Características de Seguridad

- **Zero-Knowledge Encryption** — Credenciales cifradas client-side con AES-256-GCM
- **Salt + Pepper** — Aislamiento de credenciales por dominio
- **Argon2 KDF** — Derivación de clave resistente a GPU/ASIC

- **X25519 + Ed25519** — Intercambio de claves y firmas post-cuánticas
- **Rate Limiting** — Throttling por IP
- **Security Headers** — CSP, HSTS, X-Frame-Options, X-XSS-Protection

13.2 Permisos de Chrome

La extensión solicita los mínimos permisos necesarios:

- `storage` — Para persistir vault cifrado localmente
 - `alarms` — Para verificación periódica de dominios
 - `tabs` — Para detectar cambios de URL en tiempo real
 - Host permissions — Solo para APIs de verificación de seguridad
-

Documento Generado

- **Fecha:** 2026-09-17
- **Servicios verificados:** API (healthy), PostgreSQL (healthy), Redis (healthy)
- **Build de extensión:** Completo en `dist/`
- **Estado:** Listo para instalación en Chrome